1. Enunciado

En este proyecto se pretende construir un sistema de gestión de trabajo colaborativo a través de tableros de tareas. Como referencia para identificar las características de este tipo de aplicaciones se pueden explorar aplicaciones como Trello (https://www.trello.com/).

La aplicación deberá permitir:

- Crear y modificar *tableros* donde hay *listas de tareas*. Una lista de tareas tiene *tarjetas* de manera que las tarjetas sirven para asignar tareas o para anotar información relevante.
- Las tarjetas se pueden marcar como completadas, en cuyo caso pueden pasar a una lista especial del tablero de tarjetas completadas.
- Las tarjetas pueden tener etiquetas para clasificarlas. Una etiqueta además tiene un color.
- Hay dos tipos de tarjetas: tarjetas que tienen tareas y tarjetas que tienen checklists.
- Los tableros mantienen una historia de todas las acciones de los usuarios. Por ejemplo, si se mueve una tarjeta entre listas de tareas se debe registrar una traza de que este movimiento se ha realizado.
- Es posible bloquear temporalmente un tablero para que no se puedan añadir nuevas tarjetas, solo mover tarjetas entre sus listas (por ejemplo, un tablero con TODO, DOING, DONE en el que durante una semana no se pueden añadir nuevas tarjetas).
- Para simplificar la gestión de los usuarios se utilizará una estrategia simple: cualquier persona con un correo electrónico podrá crear un tablero y obtendrá una URL única (y privada a no ser que la comparta) para su tablero.
- Un tablero puede compartirse con otros usuarios de la aplicación, para ello se comparte la URL.
- Implementar reglas a nivel de tablero o de lista de tareas:
    - Una lista no puede tener más de N ítems (configurable)
    - Una lista define que una tarjeta tiene que haber pasado por otras listas antes de llegar a ella.
- Filtrado de tarjetas por etiquetas.
- Creación de plantillas para tableros.
    - Las plantillas se definen como un fichero YAML.

Comentado [DMS1]: ¿Se puede mover una tarjeta de la lista de completadas a cualquier otra lista de nuevo?

Comentado [DMS2]: Cuando un tablero se crea, se genera automáticamente la URL. La persona que crea el tablero es la que decide si compartir o no esa URL, pero no hay que añadir restricciones del tipo “solo el propietario puede generar esa URL”.

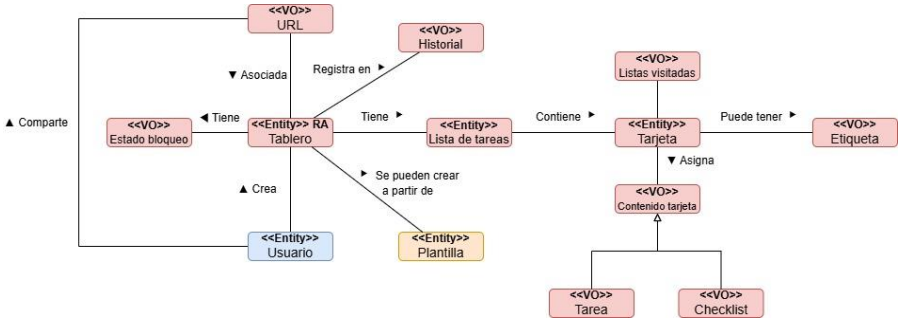
- Autenticación basada en código por correo.

2. Lenguaje ubicuo

Término	Descripción
Tablero	Espacio de trabajo en el que colocar listas de tareas
Tarea	Una acción del proyecto que puede ser asignada a una tarjeta
Lista de tareas	Lista que contiene tarjetas
Tarjeta	Componente de una lista de tareas que almacena una tarea o anota información relevante
Etiqueta	Cada etiqueta es de un color y sirve para clasificar tarjetas
Checklist	Conjunto de tareas que, una vez completo, completan la tarjeta
TODO	Representa el estado de una tarea pendiente
DOING	Representa el estado de una tarea en proceso
DONE	Representa el estado de una tarea completada
Plantilla	Fichero que permite copiar la estructura y reglas de un tablero para construir uno nuevo
Historial	Registro de actividades en el tablero

3. Modelo conceptual

Comentado [DMS3]: No sé si el agregado es muy grande aunque tenga sentido modelarlo de esta manera



4. Historias de usuario

4.1. Épica A → Gestión de tableros

HA1 – Crear tablero

Como usuario con email
Quiero crear un tablero nuevo
Para disponer de un espacio con listas donde gestionar tareas

Criterios de aceptación

* Al crear un tablero, se genera una URL única privada para el tablero
* El creador queda registrado como propietario del tablero
* Se puede inicializar el tablero vacío o desde una plantilla YAML
* Devuelve ID del tablero y URL

HA2 – Bloquear tablero

Como propietario del tablero
Quiero configurar un bloqueo temporal del tablero
Para no poder añadir nuevas tarjetas y solo moverlas entre las listas del tablero

Criterios de aceptación

* Puedo activar/desactivar bloqueo con rango de fechas (desde - hasta)
* Los cambios quedan registrados en el historial del tablero
* Solo el propietario puede cambiar el bloqueo

HA3 – Compartir tablero

Como propietario del tablero
Quiero compartir la URL del tablero con otros usuarios
Para que puedan acceder y trabajar sobre el mismo tablero

Criterios de aceptación

* Quien tenga la URL puede acceder y editar el tablero
* La URL es única para cada tablero

HA4 – Configurar prerrequisitos número de tarjetas por lista de tablero

Como propietario o editor del tablero
Quiero definir que las listas no puedan contener más de N tarjetas (configurable)
Para evitar sobrecargar el flujo de tareas y trabajo

Criterios de aceptación

* Si N se alcanza, añadir una tarjeta a la lista falla
* Si una tarjeta se va fuera de la lista y queda hueco, se puede añadir otra
* El límite puede modificarse y se registra la modificación en el historial
* La lista especial no se configura con límite N

4.2. Épica B → Listas y reglas por lista

HB1 – Crear/editar/eliminar listas

Como propietario o editor del tablero
Quiero crear, renombrar y eliminar listas dentro del tablero
Para organizar el flujo de tareas y trabajo

Criterios de aceptación

* Las listas tienen un nombre único en el tablero
* La creación, modificación y renombrado de listas se registra en el historial

HB2 – Configurar límite de N ítems en una lista

Como propietario o editor del tablero
Quiero poner un límite de tarjetas N (configurable) en una lista
Para evitar sobrecargar el flujo de tareas y trabajo

Criterios de aceptación

* Si N se alcanza, añadir una tarjeta a la lista falla
* Si una tarjeta se va fuera de la lista y queda hueco, se puede añadir otra
* El límite puede modificarse y se registra la modificación en el historial
* Intentar configurar límite N en una lista especial falla

HB3 – Configurar prerrequisitos de lista

Como propietario o editor del tablero
Quiero definir que para entrar en la lista C una tarjeta debe haber pasado por listas A y B antes
Para imponer orden en el flujo de trabajo

Criterios de aceptación

* Intentar mover una tarjeta a lista C sin haber pasado por las listas requeridas falla
* La lista de visitados de la tarjeta mantiene las listas por las que ha pasado la tarjeta

HB4 – Crear lista especial

Como propietario o editor del tablero
Quiero definir que una lista es especial de tarjetas completadas
Para reflejar un flujo ordenado y organizado de trabajos completados

Criterios de aceptación

* Si ya hay una lista especial creada anteriormente, la creación fallará
* Las tarjetas marcadas como completadas se mueven a esta lista
* El criterio a nivel de tablero de límite N no afecta a esta lista

4.3. Épica C → Tarjetas

HC1 – Crear tarjeta (tarea/checklist)

Como propietario o editor del tablero
Quiero añadir una tarjeta a una lista con contenido (tarea o checklist)
Para capturar un trabajo o información relevante

Criterios de aceptación

* Se puede crear la tarjeta como una tarea o como una checklist que contenga una lista de ítems
* Si el tablero está bloqueado para añadir, la operación falla
* La tarjeta obtiene un ID único y la creación se registra en el historial del tablero

HC2 – Mover tarjeta entre listas

Como propietario o editor del tablero
Quiero mover tarjetas entre listas
Para reflejar el progreso del trabajo

Criterios de aceptación

* Los movimientos válidos se permiten, a no ser que incumpla prerrequisitos de la lista como el límite N
* Cada movimiento genera una entrada en el historial
* Si el tablero está bloqueado, mover tarjetas está permitido

HC3 – Marcar tarjeta como completada

Como propietario o editor del tablero
Quiero marcar una tarjeta como completada
Para que pase a una lista concreta especificada

Criterios de aceptación

* La acción mueve la tarjeta a la lista especial
* Si en una checklist todos los ítems han sido marcados como completados, la tarjeta se completa automáticamente
* Si la tarjeta no ha pasado por las listas necesarias antes de pasar a la lista especial, la operación falla

* Si no existe lista especial de completadas, la operación falla y se solicita al usuario definir una lista como lista especial de completadas

HC4 – Añadir/eliminar/modificar etiquetas en tarjeta

Como propietario o editor del tablero
Quiero añadir etiquetas (color + nombre) a las tarjetas
Para poder filtrarlas y clasificarlas

Criterios de aceptación

* La etiqueta se añade individualmente a cada tarjeta
* Se pueden añadir múltiples etiquetas a una tarjeta
* Filtrado por etiqueta devuelve tarjetas que contengan esa etiqueta
* La creación, eliminación o modificación de las etiquetas se ve reflejada en el historial

Comentado [DMS4]: Para el filtrado de etiquetas, no sé si será más apropiado modelarlo como entity o value object

HC5 – Editar contenido de la tarjeta

Como propietario o editor del tablero
Quiero editar la descripción de la tarea o modificar la checklist de ítems
Para mantener la información actualizada

Criterios de aceptación

* Se reemplaza el contenido de la tarjeta
* Cada edición genera un evento en el historial

4.4. Épica D → Plantillas

HD1 – Definir plantilla en YAML

Como propietario o editor del tablero
Quiero crear una plantilla (fichero YAML) que describa listas iniciales y sus reglas
Para reutilizar configuraciones de tablero

Criterios de aceptación

* La plantilla incluye un nombre, las mismas listas y reglas por lista que en el tablero que se ha creado la plantilla
* El YAML se valida, error si no cumple el formato

HD2 – Crear tablero desde plantilla

Como usuario
Quiero crear un tablero a partir de una plantilla YAML
Para iniciar rápidamente la estructura con reglas

Criterios de aceptación

* El tablero se crea con listas y reglas tal y como define la plantilla
* En el historial se registra que el tablero se ha creado usando la plantilla

4.5. Épica E → Autenticación y seguridad

HE1 – Login por código email

Como usuario con correo
Quiero iniciar sesión introduciendo mi email y un código que me llega por correo
Para no gestionar contraseñas

Criterios de aceptación

* La petición de login genera un código temporal enviado por email
* El código expira pasado cierto tiempo

Comentado [DMS5]: TBD el tiempo, porque aún no sabemos especificar el tiempo, y si los métodos de programación nos lo permiten. No afectará al diseño del proyecto, por eso queda ambiguo.

4.6. Épica F → Historial

HF1 – Registrar eventos en el historial del tablero

Como propietario o editor del tablero
Quiero registrar en un historial las acciones sobre el tablero
Para monitorizar cambios y ver quién ha hecho cada acción

Criterios de aceptación

* Cada acción generará un registro con un tipo.
* Cada registro lleva un timestamp y los detalles del tipo de registro. Formato del registro ejemplo: tipo, usuario (email), timestamp, detalles (e.g. CardMoved cardId fromListId toListId)

4.7. Épica G → Búsqueda y filtrado

HG1 – Filtrar tarjetas por etiqueta

Como propietario o editor del tablero
Quiero ver solamente tarjetas que contienen X etiquetas
Para centrarme en un subconjunto de trabajo

Criterios de aceptación

* Se puede filtrar por una o varias etiquetas a la vez (AND/OR configurable)
* No aparecen tarjetas que no cumplan con los filtros de la búsqueda

4.8. Épica H → Reglas de negocio y validaciones (historias técnicas)

HH1 – Validar límite N al añadir/mover tarjetas

Como sistema
Quiero impedir acciones que violen el límite N de una lista
Para mantener el invariante de capacidad

Criterios de aceptación

* Las operaciones de añadir tarjeta y mover tarjeta comprueban el límite antes de aplicarse
* Si se incumple la regla, se lanza una excepción de dominio

HH2 – Validar prerrequisitos de listas al mover tarjeta

Como sistema
Quiero comprobar las listas visitadas de la tarjeta
Para validar prerrequisitos y garantizar el flujo definido

Criterios de aceptación

* La operación de mover tarjeta comprueba las listas ya recorridas por esa misma tarjeta
* Si no se cumple, la operación se rechaza con un mensaje que indica qué listas faltan

5. Reglas de negocio

- R1 – Cuando un tablero está bloqueado, durante ese tiempo no se pueden añadir nuevas tarjetas, solo moverlas entre sus listas
- R2 – Cuando una lista está configurada con límite de ítems N, no se pueden añadir ni mover nuevas tarjetas a esta lista
- R3 – Si una lista define que una tarjeta tiene que haber pasado por otras listas antes de llegar a ella, solo aceptará tarjetas que cumplan dicho prerrequisito
- R4 – Si existe lista especial, marcar tarjeta como completada mueve la tarjeta a esa lista. Si no existe, la operación falla y se solicita al usuario marcar una lista como lista especial de completadas

6. Entidades y objetos valor

Definimos entidades a aquellos componentes del dominio que tienen identidad propia y que pueden mutar a lo largo de su ciclo de vida. Por otro lado, el value object se define por sus atributos y no tiene identidad propia, comúnmente tratándose como objetos inmutables y sustituyéndolos por otros nuevos en caso de ser modificados. Con esta definición, categorizamos cada elemento de nuestro dominio:

Comentado [DMS6]: Así tendremos que modelar la mayoría si no todos los elementos del dominio clasificados como value objects

- **Tablero:** Entidad, puesto que tiene identidad propia (nombre, usuario creador, URL) y puede ir cambiando a lo largo de su ciclo de vida.
- **Lista de tareas:** Entidad, ya que también tiene identidad propia y cada lista puede ser modificada. Tiene reglas propias asociadas y necesitamos distinguir entre unas listas y otras.
- **Tarjeta:** Entidad, ya que puede haber dos tarjetas con el mismo contenido y etiquetas y aún así ser distintas, tiene identidad.
- **Usuario:** Entidad por definición, tienen identidad, se identifican por el email.
- **Plantilla:** Entidad, porque cada plantilla es diferente, ya que se crea mediante un fichero YAML que copia la estructura de cada entidad Tablero.
- **URL:** Value object, no tiene identidad propia en el dominio, solo importa el valor del enlace en sí.
- **Estado bloqueo:** Value object, simplemente importan sus atributos de saber si está activo y la fecha (desde cuándo hasta cuándo).
- **Historial:** Value object en el modelo conceptual, será un fichero append-only en el que se introduzcan entradas inmutables que no tienen identidad.
- **Listas visitadas:** Value object, no tiene identidad, es solamente un conjunto de valores inmutable que se utiliza para validar prerrequisitos.
- **Etiqueta:** Value object, importa solamente por su nombre y su color, son los atributos que observamos, no necesita de identidad propia. En el caso de considerar un catálogo de etiquetas disponible y gestionable, sí se podría tratar como una entidad, pero no lo modelaremos así.
- **Contenido tarjeta:** Value object, es simplemente el contenido dentro de la tarjeta, puede ser igual en varias tarjetas. No necesita identidad.
- **Tarea:** Value object, es un tipo concreto de Contenido tarjeta, vive dentro de cada tarjeta.
- **Checklist:** Value object, igual que la tarea, es un subtipo de Contenido tarjeta. Vive dentro de cada tarjeta y carece de identidad.

7. Agregados

El concepto de agregado se define como un conjunto de objetos del dominio que se tratan como una unidad de consistencia y cuya coherencia se protege mediante reglas del dominio.

Nuestro DDD consta de 3 agregados que cobran sentido, mantienen la coherencia y preservan las reglas del dominio al ser agrupados:

- **Agregado Tablero – Tablero como raíz del agregado →** A pesar de que este agregado sea grande, tiene sentido que abarque todos esos elementos del dominio en la aplicación que queremos construir. Las invariantes tienen en cuenta las listas y las tarjetas que están contenidas en cada tablero. En otras palabras, las reglas más importantes necesitan leer o modificar varias partes del dominio a la vez. Al estar todo dentro de la misma raíz, las invariantes se garantizan mejor. Además, de esta manera conseguimos dentro del agregado consistencia fuerte entre operaciones. En cuanto a la concurrencia, si dos usuarios mueven o añaden tarjetas a la vez, este agregado es el punto natural del que partir para evitar romper reglas como exceder el límite de N elementos, evitar movimientos inválidos entre listas o cualquier otra acción incoherente.

- **Agregado Usuario – Usuario como raíz del agregado →** Su identidad y su ciclo de vida es propio e independiente al tablero. Un mismo usuario puede crear o acceder a varios tableros, lo que lo hace independiente. Además, tiene reglas independientes del tablero asociadas a él, como el acceso al login mediante un código de autenticación. Se hace referencia a él desde otros agregados mediante el correo electrónico del usuario.

- **Agregado Plantilla – Plantilla como raíz del agregado →** La plantilla es una entidad reutilizable que es **independiente** del tablero. La plantilla se crea una vez y se puede usar para crear **muchos tableros**, con las mismas listas y reglas, pero diferentes en esencia y con ciclos de vida distintos. También sigue sus propias invariantes, como el cumplimiento del esquema correcto del YAML. La coherencia entre las reglas de un tablero y la plantilla serán **heredadas** del propio tablero, pero pasarán a formar parte de las características de la plantilla. Dicho de otra forma, se copia la estructura del tablero pero se desentiende de aquel tablero a partir del cual se ha creado la plantilla.

8. Servicios de dominio

Los servicios de dominio contienen lógica que no pertenece exclusivamente a una sola entidad o value object o que orquesta varias responsabilidades.

- ServicioGestionTablero → Maneja las operaciones complejas sobre el agregado tablero. Aunque muchos de estos métodos se ejecutan en métodos dentro del propio agregado, este servicio de dominio actúa como fachada entre casos de uso (valida precondiciones de aplicación, inicia la transacción, invoca métodos del agregado, llama al logger del historial…)

```
public class ServicioGestionTablero {
	TableroId crearTablero(CrearTableroCmd cmd)
	void bloquearTablero(TableroId id, EspecBloqueo espec, Actor actor)
	void desbloquearTablero(TableroId id, Actor actor)
	TarjetaId addTarjeta(TableroId id, ListaId listaId, ContenidoTarjeta
contenido, Actor actor)
	void moverTarjeta(TableroId id, TarjetaId id, ListId listaDestino, Actor
actor)
	void completarTarjeta(TableroId id, TarjetaId id, Actor actor)
	void configurarLimiteLista(TableroId id, ListaId listaId, int limite,
Actor actor)
	void configurarPrerrequisitosLista(TableroId id, ListaId listaId,
List<ListaId> prerrequisitos, Actor actor)
}
```

- ServicioPlantilla → Responsable de validar y parsear el fichero YAML, guardar plantillas y crear tableros a partir de plantillas.

```
public class ServicioPlantilla {
	// Valida el esquema
	PlantillaId crearPlantilla(String yaml, Actor actor)

	// Devuelve la estructura que utiliza ServicioGestionTablero para crear
el tablero
	EspecsTablero parsearPlantilla(PlantillaId)
}
```

- ServicioHistorial → Simplemente se encarga de añadir las entradas al historial del tablero.

Comentado [DMS7]: ¿Deberiamos introducir métodos de consulta o simplemente que sea un fichero log que genere?

Comentado [DMS8]: Documentar también acerca de la persistencia en el historial, con JPA y todo eso.

```
public class ServicioHistorial {
	void append(EntryHistorial entry)
}
```

- ServicioAutenticacion → Genera y valida los códigos de login y los envía por correo.

```
public class ServicioAutenticacion {
	void enviarCodigoLogin(Email email)
	AuthToken verificarCodigoLogin(Email email, Codigo codigo)
}
```

No forma parte del DDD pero será necesario documentar:

9. Puertos / Repositorios (Interfaces de salida)

Comentado [DMS9]: ChatGPT

Definiremos estas interfaces en la capa de dominio/aplicación (puertos):

```
interface RepositorioTableros {
	void guardar(Tablero tablero)
}

interface RepositorioPlantilla {
	void guardar(Plantilla plantilla)
}

interface RepositorioUsuarios {
	void guardar(Usuario usuario)
}

interface PuertoRegistroHistorial {
	void append(EntryHistorial entry)
}

interface PuertoEnvioEmail {
	void enviarEmail(MensajeEmail m)
}

interface PuertoParserYAML {
	YamlCorrecto parse(String yaml) throws PlantillaInvalida
}
```

Esto nos permitirá implementar los adapters concretos en Spring Boot.

10. Eventos del dominio

Cuando ocurran, publicar los siguientes eventos que podrán agregarse en el historial, pueden ser las entries del historial.

```
public class EventosDominio {
	TableroCreado(tableroId, actorEmail, desdePlantilla?)
	TarjetaCreada(tableroId, tarjetaId, listaId, actorEmail)
	TarjetaMovida(tableroId, tarjetaId, fromListaId, toListaId, actorEmail)
	TarjetaCompletada(tableroId, tarjetaId, actorEmail)
	TableroBloqueado(tableroId, especBloqueo, actorEmail)
	PlantillaCreada(plantillaId, actorEmail)
}
```

11. Servicios de aplicación (contratos REST)

Comentado [DMS10]: ChatGPT

Cada caso de uso importante tendrá un servicio de aplicación que será invocado por un controlador REST. En Spring Boot, los controladores serán adaptadores de entrada y los

servicios ServicioGestionTablero y ServicioAutenticacion serán servicios de aplicación que dependen de puertos (repositorios, email, etc).

Algunos ejemplos:

- `POST /boards` → `BoardController.create()` → llama `BoardManagementService.createBoard(CreateBoardCmd)`
  - body: `{ "name": "...", "fromTemplateId": "..." }`
  - response: `{ "boardId", "shareUrl" }`
- `POST /boards/{boardId}/lists` → crear lista
- `POST /boards/{boardId}/lists/{listId}/cards` → crear tarjeta (body con CardContent)
- `POST /boards/{boardId}/cards/{cardId}/move` → mover tarjeta (body: `toListId`, actorEmail)
- `POST /auth/requestCode` → `AuthService.sendLoginCode(email)`
- `POST /auth/verifyCode` → `AuthService.verifyLoginCode(...)`
- `GET /boards/{boardId}/activities?size=20&page=0` → `ActivityLogService.query(...)`